

Avaliação Individual — Versão A: Refatorando a Oficina de Conserto

Disciplina	Programação para Servidores Web (PSI)
Modalidade	Individual
Valor	55,0 pontos
Duração	1h30 (90 minutos)

Regras

É permitido:

- Consultar a documentação oficial do Flask e do Jinja2;
- Consultar os materiais, slides e códigos desenvolvidos **nas aulas desta disciplina**;
- Consultar suas próprias anotações.

É terminantemente proibido:

- Usar qualquer ferramenta de **IA generativa** (ChatGPT, Copilot, Gemini, Claude, DeepSeek ou similar) — inclusive para “tirar dúvidas” ou “gerar ideias”;
- Consultar, copiar ou receber código de colegas, presencialmente ou por qualquer meio;
- Usar código pronto da internet (Stack Overflow, tutoriais, repositórios);
- Usar bancos de dados (SQLAlchemy, SQLite, persistência) ou qualquer extensão que não seja o Flask padrão.

O descumprimento de qualquer item acima implica **nota zero** e registro conforme o regimento da instituição. O professor poderá solicitar **defesa oral** do código: você deverá explicar qualquer trecho do seu próprio trabalho.

Se o tempo acabar, suba o que tiver pronto. A correção considera o progresso parcial.

Entrega

A entrega é feita pelo **GitHub usando apenas o navegador**:

1. Acesse <https://github.com> e faça login;
2. No canto superior direito, clique no **+** e depois em **New repository**;
3. Nome do repositório: `psi-avaliacao2-seunome` (minúsculas, sem espaços);
4. Marque **Public** e clique em **Create repository**;
5. Na tela do repositório vazio, clique em **uploading an existing file** (ou *Add file → Upload files*);

6. **Arraste a pasta inteira do projeto** para a área de upload — arrastar a pasta preserva a estrutura de diretórios;
 7. Clique em **Commit changes**;
 8. Confira se as pastas aparecem no repositório;
 9. Copie o link do repositório e entregue ao professor até o fim da aula.
-

Contexto

A **Oficina de Conserto** do campus repara aparelhos eletrônicos, mecânicos e elétricos. O responsável contratou um programador, que entregou uma primeira versão do sistema. A aplicação **funciona parcialmente**, mas foi escrita toda em um único arquivo, sem separação entre as camadas. Você foi contratado para **reorganizar o projeto** seguindo o padrão **MVC com Blueprints**, como visto em aula.

O que importa nesta avaliação é a **organização da arquitetura**. Não use banco de dados: todos os dados continuam em memória (listas/dicionários).

Situação-problema

O projeto inicial possui **vários problemas arquiteturais** que não são apenas erros de sintaxe. Por exemplo:

- rotas concentradas no lugar errado;
- a classe `Servico` (que deveria ser a camada Model) definida **dentro do Controller**;
- um Blueprint criado mas **não registrado** na aplicação;
- responsabilidades de apresentação (HTML) misturadas com o Controller;
- código repetido para filtrar serviços;
- endpoint utilizado incorretamente.

Sua tarefa é **ler o projeto, identificar os problemas e reorganizá-lo** para que fique no padrão MVC com Blueprints e volte a funcionar navegando do início ao fim.

Objetivo

Reorganizar o projeto inicial para a arquitetura **MVC com Blueprints**, separando corretamente Model, View e Controller, e deixar a aplicação **funcionando após a refatoração**.

Requisitos obrigatórios

1. **Identifique e liste** os problemas arquiteturais do código inicial (registre no `README.md`, citando os trechos do código original);

2. **Separe corretamente** as camadas Model, View e Controller;
3. **Defina a classe** `Servico` com seus atributos na camada Model (`models.py`) — somente a classe e os atributos, sem banco de dados;
4. **Crie/reorganize** os Blueprints da aplicação (ex.: `servicos`, `autenticacao`), cada um com `__init__.py` e `routes.py`;
5. **Mova as rotas** para o Controller (blueprint) adequado, usando `@servicos_bp.route`, `@auth_bp.route`, etc.;
6. **Registre os Blueprints** no `app.py`, deixando-o enxuto;
7. **Organize os templates** nas pastas dos respectivos blueprints (ex.: `blueprints/servicos/templates/servicos/`);
8. **Passe os dados** do Controller para a View usando `render_template`;
9. **Ajuste** `url_for`, **imports** e **endpoints** onde necessário;
10. **Deixe a aplicação funcionando** após a refatoração: listar serviços, buscar, filtrar por categoria, ver detalhes, entrar e sair do sistema.

Estrutura inicial fornecida

```
oficina/
├── app.py                ← Controller (tudo misturado aqui)
├── models.py            ← Model (dados em memória)
├── blueprints/
│   ├── servicos/
│   │   ├── __init__.py  ← Blueprint criado, mas NÃO registrado
│   │   └── routes.py    ← ainda não usado corretamente pela aplicação
├── templates/
│   ├── base.html
│   ├── index.html
│   └── login.html
```

Arquivos que você criará ou modificará

- `app.py` — deve passar a registrar os blueprints;
- `models.py` — deve conter a classe `Servico` e seus atributos;
- `blueprints/servicos/routes.py` — rotas corrigidas usando o blueprint;
- `blueprints/auth/` (novo) — rotas de login/logout no padrão de blueprint;
- `templates/` — reorganizados nas pastas dos blueprints
(`blueprints/<nome>/templates/<nome>/`);
- `README.md` (novo) — lista de problemas identificados + respostas da defesa escrita.

A forma de organizar os blueprints (quais criar, quais rotas colocar em cada um) é parte da avaliação. Não há comando único obrigatório, mas todos os blueprints precisam estar registrados

no `app.py`.

Defesa escrita (README.md)

Responda **com suas palavras** e **citando trechos do seu código ou do código inicial**:

1. Liste **pelo menos 4 problemas arquiteturais** que você encontrou no código inicial e explique por que cada um viola o padrão MVC.
2. Onde ficou a camada Model no seu projeto? Onde ficaram os Controllers? Cite um trecho de cada.
3. Por que o `url_for` e os endpoints precisaram ser ajustados durante a refatoração? Cite um exemplo de mudança que você fez.

Pontuação

Bloco	Conteúdo avaliado	Pontos
MVC	Identificação dos problemas (defesa escrita) + separação correta das camadas	10,0
Model	Classe <code>Servico</code> definida com seus atributos em <code>models.py</code>	5,0
Controller e rotas	Rotas nos controllers (blueprints), uso de <code>render_template</code> e da camada Model	15,0
Blueprint e integração	Criação/reorganização dos blueprints + registro no <code>app.py</code>	10,0
Views/templates	Templates organizados, herança de <code>base.html</code> , <code>url_for</code> e passagem de dados	10,0
Funcionamento geral	Aplicação executando após a refatoração + organização do projeto	5,0
Total		55,0

Checklist final antes de subir

- ☐ `app.py` registra os blueprints e está enxuto
- ☐ `models.py` contém a classe `Servico` com seus atributos
- ☐ Rotas usam `@<blueprint>_bp.route` e foram movidas para os controllers
- ☐ Templates ficam nas pastas dos blueprints e herdam de `base.html`
- ☐ Links usam `url_for` com o prefixo do blueprint
- ☐ Login, listagem, busca, categoria e detalhe funcionam no navegador
- ☐ Nenhum banco de dados foi usado
- ☐ `README.md` criado com as 3 perguntas respondidas

- ☐ Repositório no GitHub com o link enviado ao professor